



COMPUTATIONAL BIOLOGY

F2 REPORT

From 2D to 3D: Nuclei Segmentation

Using ImageJ and Python

Student name: Thai Trieu Ngoc Diep

Matriculation number: 3034398

Supervisor: Prof. Dr. Sabine Fischer

June /2026

Table of Contents

Abstract	1
1. Introduction.....	2
2. Methods	3
2.1. Dataset description	3
2.2. Software and packages	4
2.3. Ground truth annotation	4
2.4. Evaluation metrics.....	4
3. Results.....	5
3.1. ImageJ-based segmentation workflow	5
3.2. Python-based segmentation pipeline	5
3.3. 2D mouse brain nuclei segmentation	6
3.4. Evaluation of 2D nuclei segmentation	8
3.5. 3D mouse blastocyst segmentation	9
3.6. Evaluation of 3D mouse blastocyst segmentation.	11
4. Discussion.....	12
5. Conclusion	14
Code availability.....	14
Acknowledgment	14
References.....	15

Abstract

Background and Objective: Nuclei segmentation is a fundamental step in fluorescence microscopy image analysis and is required for many downstream quantitative applications, including cell counting and morphological characterization. Despite the growing use of deep learning methods, classical image-processing approaches remain widely used because of their simplicity, interpretability, and computational efficiency. This project aims to develop, evaluate, and compare classical image-processing workflows for 2D and 3D nuclei segmentation.

Methods: Fluorescence microscopy datasets from mouse brain tissue and a mouse blastocyst were analyzed using Fiji/ImageJ and a custom Python pipeline. The workflow combined Gaussian filtering, Otsu thresholding, morphological operations, distance-transform-based marker detection, and watershed segmentation. Manual ground truth annotations were generated for evaluation. Segmentation performance was assessed using pixel-wise precision, recall, and F1 score.

Results: In the 2D mouse brain dataset, segmentation quality was strongly influenced by the Gaussian smoothing parameter, with $\sigma = 1$ providing the best balance between noise reduction and preservation of nuclear boundaries. Morphological processing further reduced over-segmentation. In the 3D blastocyst dataset, watershed segmentation successfully separated connected nuclei, resulting in 148 segmented nuclei. Quantitative evaluation against manual annotations yielded a precision of 0.686, recall of 0.913, and an F1 score of 0.783. Compared with the ImageJ 3D Suite workflow, the custom Python pipeline produced more coherent nuclear objects and reduced over-segmentation.

Conclusions: This study demonstrates that classical image-processing methods can effectively segment nuclei in both 2D and 3D fluorescence microscopy images when appropriate parameters are applied. The custom Python pipeline improved segmentation consistency compared to a standard ImageJ workflow, particularly in 3D datasets. However, challenges such as over-segmentation, difficulties in densely packed regions, and imaging artifacts highlight the limitations of classical approaches, suggesting potential benefits from more advanced machine learning-based for future work.

1. Introduction

Fluorescence microscopy is an essential tool used in biological research to visualize cellular structures and investigate biological processes at the cellular level [1]. Fluorescent markers enable researchers to study the dynamics of subcellular biology and its complex mechanisms at higher spatial and temporal resolutions [1]. Among cellular structures, cell nucleus is vital component because its morphology and spatial features act as indicators across cellular states such as cell differentiation, proliferation, and pathological conditions [2-4]. Therefore, accurate identification of cell nuclei in fluorescence microscopy images, commonly through DAPI staining, is a critical step in quantitative biological image analysis. Moreover, segmentation of cells and nuclei is a prerequisite for downstream analysis processes, such as counting and tracking [5].

However, accurate nuclei segmentation remains a challenging problem in fluorescence microscopy. Fluorescence images often contain noise, uneven illumination, intensity variation, and densely packed nuclei [6]. In addition, optical microscopy is fundamentally limited by diffraction, where the point spread function (PSF) introduces spatial blurring that causes neighboring nuclei to appear merged or connected [7]. These difficulties become more pronounced in 3D image volumes, where nuclei extend across multiple z-slices and microscopic data often exhibits anisotropic voxel spacing, resulting in different spatial resolutions along the axial and lateral directions [8]. Consequently, numerous segmentation approaches have been developed to design and improve accurate nuclear and cellular segmentation, ranging from classical algorithms to machine learning and deep learning methods [9]. Although deep learning methods have demonstrated strong segmentation performance, classical image processing approaches remain widely used due to their effectiveness, simplicity of use, and computational efficiency.

The aim of this project is to apply and evaluate a computational workflow for 2D and 3D nuclei segmentation on fluorescence microscopy datasets of mouse brain and mouse embryo samples using classical image processing methods. The workflow was implemented using both ImageJ2/Fiji [10, 11] graphical tools and a custom Python pipeline. The Python-based workflow combined Gaussian filtering [12], Otsu thresholding [13], morphological operations, and watershed segmentation to isolate individual nuclei. To evaluate performance, ground truth annotations were manually generated, and the predicted masks were compared against these annotations using pixel-level metrics, including precision, recall, and the F1-score.

2. Methods

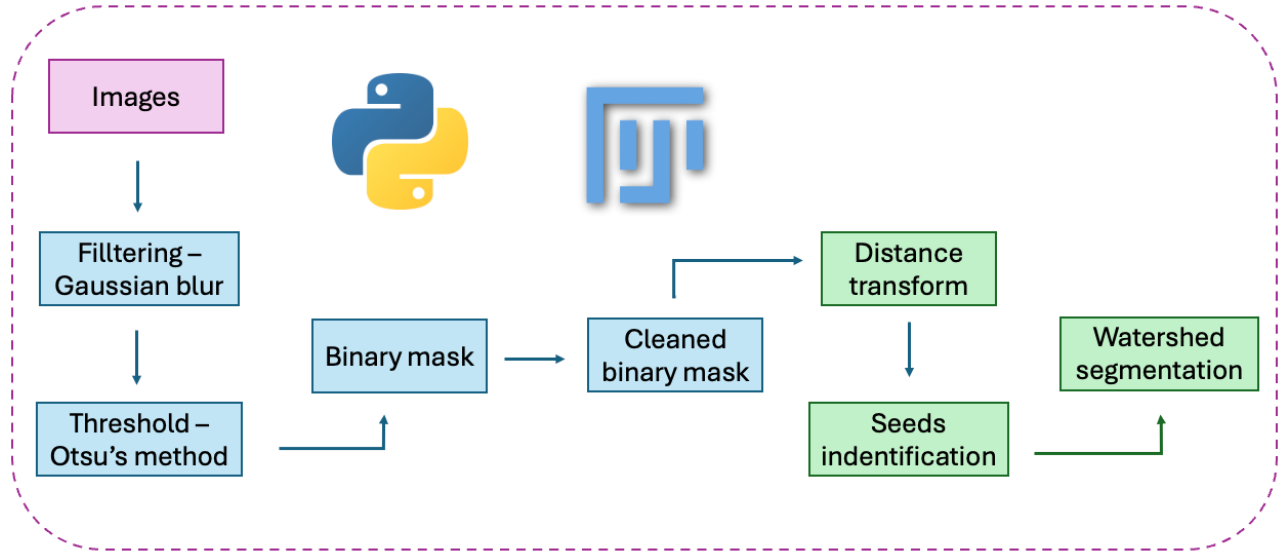


Figure 1: Flowchart of nuclei segmentation pipeline used in the current study and implemented in both Python and Fiji.

2.1. Dataset description

The first dataset consisted of fluorescence microscopy images of 171 selected tiles from the right hemisphere of a mouse brain, which is publicly available from the BioImage Archive under accession number S-BSST700. Each tile had dimensions of 1000×1000 pixels and contained six imaging channels, including one nuclei channel, one anchor channel, and four coding channels corresponding to different molecular markers. The channels correspond to an anchor probe (Atto_490LS), nuclear staining with DAPI, and four nucleotide-specific probes: A (Atto_425), C (Alexa_488), G (Alexa_568), and T (Alexa_647). Four sequencing rounds were performed for each tile.

The second dataset consisted of a three-dimensional fluorescence microscopy image of a mouse embryo stored in .lsm format and provided by the AI in Computational Biology group at the University of Würzburg. The DAPI channel image volume had dimensions of $78 \times 512 \times 512$ voxels ($z \times y \times x$), corresponding to a stack of 78 slices with a resolution of 512×512 pixels per slice. The voxel spacing was anisotropic, with a resolution of $1.0000 \mu\text{m}$ along the z-axis and $0.3946 \mu\text{m}$ in the x- and y-directions.

2.2. *Software and packages*

The automatic nuclei segmentation was implemented in Python using Jupyter Notebook and executed within Visual Studio Code (VS Code), and the semi-automatic one was conducted in ImageJ2.

Table 1: Software packages and libraries utilized in the Python pipeline.

Software	Version	Function
Python	3.12	Automatic nuclei segmentation
ImageJ2 [10]	2.16.0	Semi-automatic nuclei segmentation
Packages	Version	Function
scikit-image [14]	0.26.0	Image processing, visualization
SciPy [15]	1.17.0	Image processing
NumPy [16]	2.4.4	Arrays handling
Napari [17]	0.6.5	Image visualization

2.3. *Ground truth annotation*

Manual annotation was performed using the Segmentation Editor plugin in ImageJ. A small region of the image was cropped to simplify the annotation process and converted to grayscale for easier visual interpretation. Each nucleus was manually marked across a few slices and subsequently interpolated throughout the volume. Individual nuclei were assigned a unique label identifier to ensure distinct labelling for evaluation of segmentation performance later.

2.4. *Evaluation metrics*

Segmentation performance was evaluated by comparing predicted binary masks with ground truth annotations on a slice-by-slice across the 78 z-slices. For each slice, both predicted and ground truth masks were binarized such that all non-zero pixels were set to foreground. Pixel-wise true positives (TP), corresponding to correctly predicted foreground pixel, false positives (FP), corresponding to predicted foreground pixels do not present in ground truth, and false negatives (FN) where ground truth foreground pixels were missed by the prediction, all of which were counted

accumulated across all slices over all slices of the 3D the volume. From these values, standard evaluation metrics were computed:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (1)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2)$$

$$F1 = \frac{2 * \text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}} \quad (3)$$

Ranging from 0 to 1, a higher F1 score indicates better segmentation overlap between the predicted mask and the ground truth, thereby providing a quantitative assessment of segmentation performance.

3. Results

3.1. *ImageJ-based segmentation workflow*

2D images were analyzed using Fiji distribution of ImageJ2 [10, 11]. Images were first smoothed using Gaussian Blur ($\sigma=1$) [12]. Nuclei were then segmented by applying Otsu thresholding [13] via Image → Adjust → Threshold. The resulting thresholded images were converted to binary masks using Process → Binary → Make Binary, and touching nuclei were separated using Process → Binary → Watershed. Segmented nuclei were quantified using Analyze Particles, with the size threshold set to 100–infinity pixels² to exclude debris and Show set to Outlines.

For 3D images, the same general workflow was applied using the 3D Suite plugin and 3D Object Counter for segmentation and quantification.

3.2. *Python-based segmentation pipeline*

To establish an automated, reproducible, and parameter-controlled analysis, nuclei were segmented using a Python-based workflow (3.12). Images were first smoothed with Gaussian filtering, then binarized using Otsu thresholding. Morphological cleanup was applied to remove small objects, after which a distance transform was computed to identify markers from local maxima. These markers were used in a watershed algorithm to separate touching nuclei. The workflow was implemented using the software packages listed in Table 1, and Napari was used for visualization.

3.3. 2D mouse brain nuclei segmentation

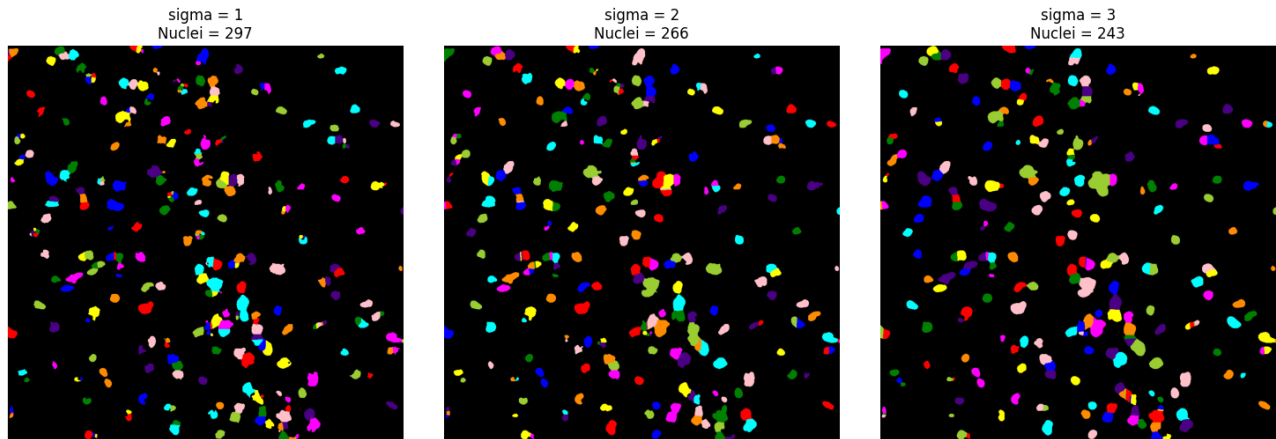


Figure 2: Effect of Gaussian smoothing (sigma - σ) on nuclei segmented results using Python. Increasing σ leads to increased under-segmentation.

The result in mouse brain nuclei dataset in Figure 2 shows that Gaussian smoothing had a pronounced effect on the nuclei segmentation results. Since σ controls the degree of smoothing, increasing σ resulted in stronger blurring of nuclei boundaries. As a result, adjacent nuclei became less distinguishable and were more likely to be merged into a single object, leading to fewer detected nuclei and increased under-segmentation. Among the tested values, $\sigma = 1$ displayed the best final segmentation results by balancing noise reduction and boundary preservation and was therefore chosen as the default parameter for all downstream analyses. However, some over-segmentation was still observed at certain positions, which cannot be resolved by adjusting the smoothing parameter alone, suggesting that optimization of additional segmentation parameters is necessary to further improve accuracy.

To address the over-segmentation observed after Gaussian smoothing, morphological processing using erosion and dilation was subsequently applied to refine segmented objects and reduce segmentation artifacts. As shown in Figure 3, the upper row displays the initial segmentation workflow, which identified a total of 297 nuclei. To improve the segmentation results, isotropic erosion with a radius of 3 pixels was applied first, followed by isotropic dilation with a radius of 4 pixels. This refinement step removed small irregular structures and helped smooth object boundaries, allowing fragmented regions to merged more coherently into individual nuclear masks.

As a result, the number of detected nuclei decreased to 250, indicating a reduction in over-segmentation.

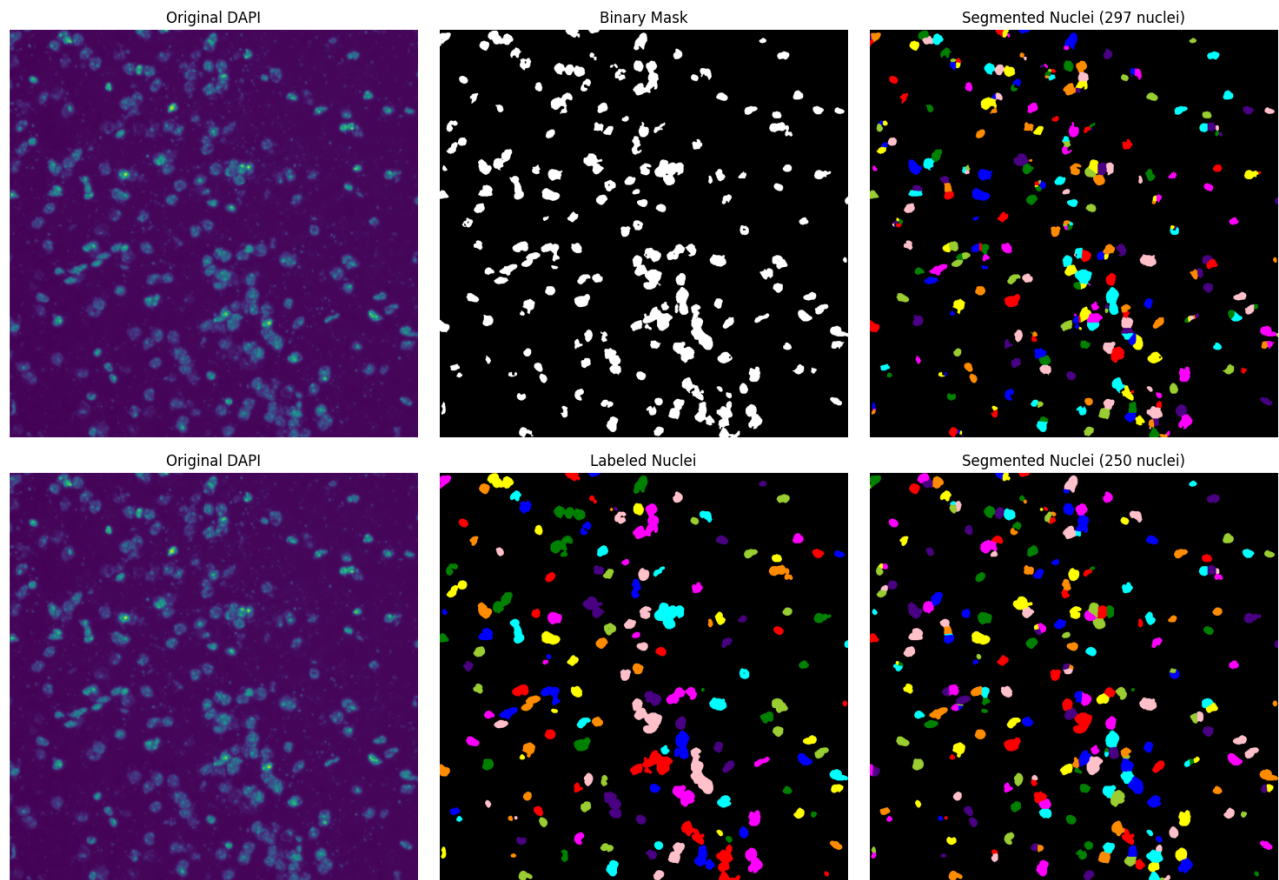


Figure 3: Improved nuclei segmentation following morphological processing using Python.

The lower row (with morphological processing) shows reduced over-segmentation compared to the upper row (without morphological processing).

3.4. Evaluation of 2D nuclei segmentation

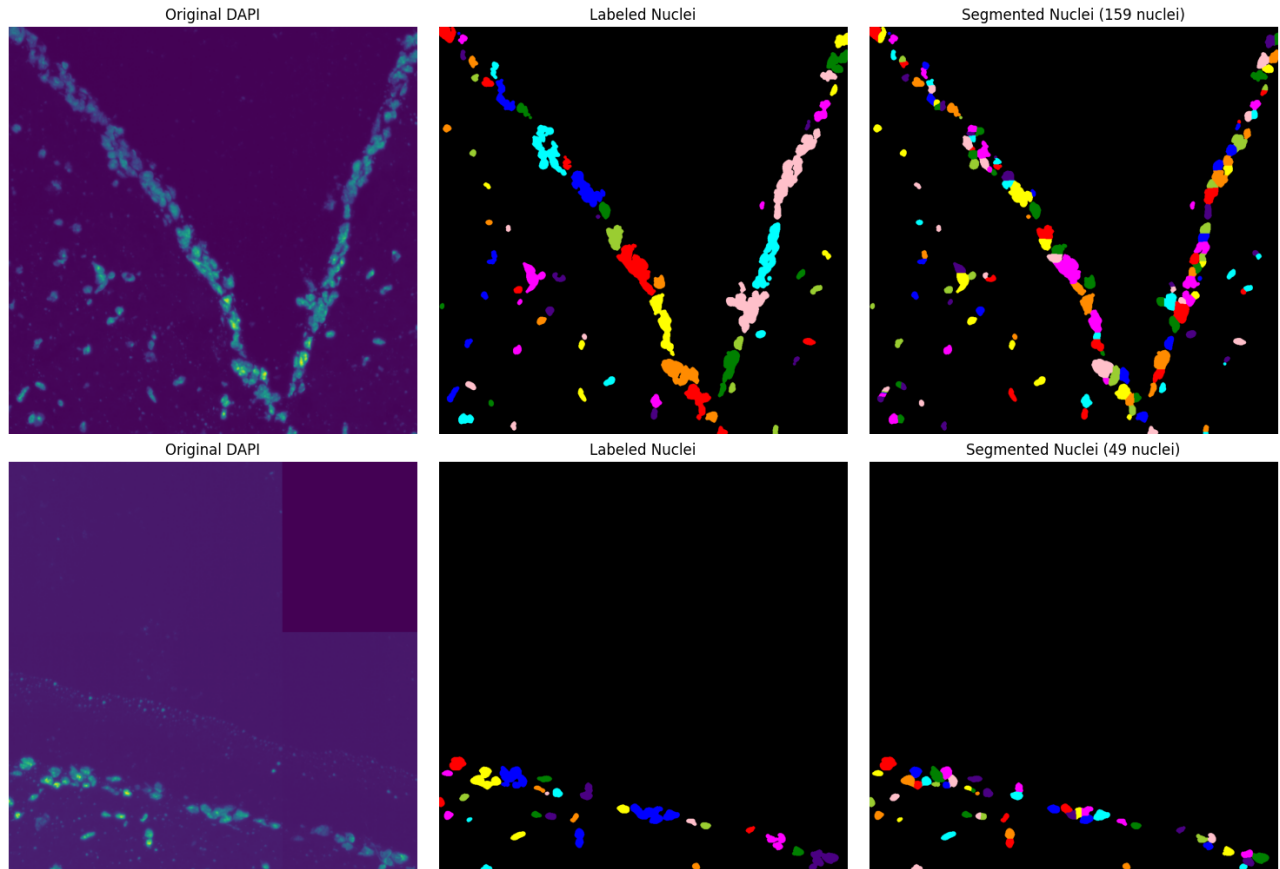


Figure 4: Evaluation of the processing pipeline and its parameter settings in different fields of view using Python.

In order to assess the robustness and accuracy of the current algorithm, additional fields of view (FOVs) with varying nuclei densities were analyzed. The lower row of Figure 4 is shown because it contains fewer nuclei, allowing easier verification of the segmentation by eye. In this region, the algorithm detected 49 nuclei, which is close to the 45 nuclei observed in the original DAPI image. However, in the upper row, the nuclei are tightly clustered, arranging in a V-shaped pattern in the left side and no longer have a regular circular shape, which reduces segmentation accuracy, highlighting the limitations of the current approach when applied to densely clustered nuclei.

3.5. 3D mouse blastocyst segmentation

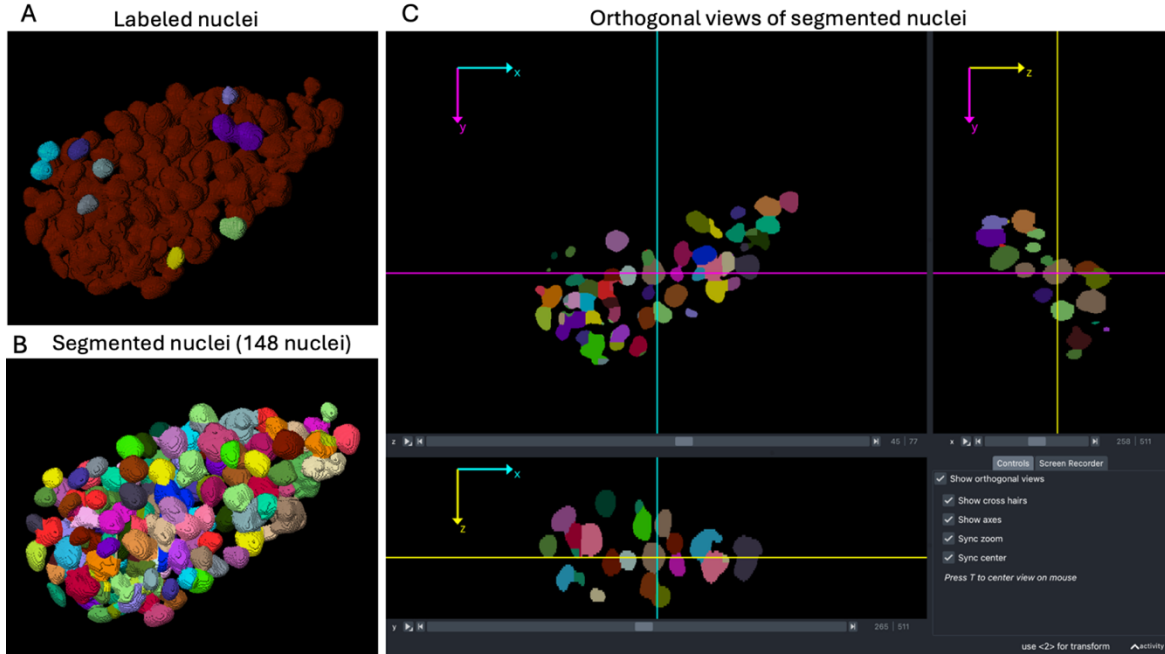


Figure 5: Automatic 3D nuclei labeling of a blastocyst (A), watershed segmentation result (B), and orthogonal views of the segmented volume (C), visualized in Napari using Python.

For mouse embryo dataset, as shown in Figure 5A, labeling of the binary mask initially resulted in only a small number of connected components (approximately nine labels). Most nuclei were assigned to a single large, connected component, represented by the large brown region. This occurred because adjacent nuclei remained connected across multiple z-slices. Although individual nuclei could often be distinguished within a single 2D slice, weak intensity bridges between neighboring nuclei persisted throughout the z-stack, causing them to be interpreted as one continuous 3D object during connected-component labeling. To separate these connected structures, watershed segmentation was applied after that. This procedure successfully divided the large, connected component into individual nuclei, resulting in 148 segmented nuclei (Figure 5B). The orthogonal views shown in Figure 5C provide cross-sectional visualization of the segmented volume in the XY, XZ, and YZ planes. These views allow assessment of segmentation quality throughout the entire 3D volume rather than within individual slices alone.

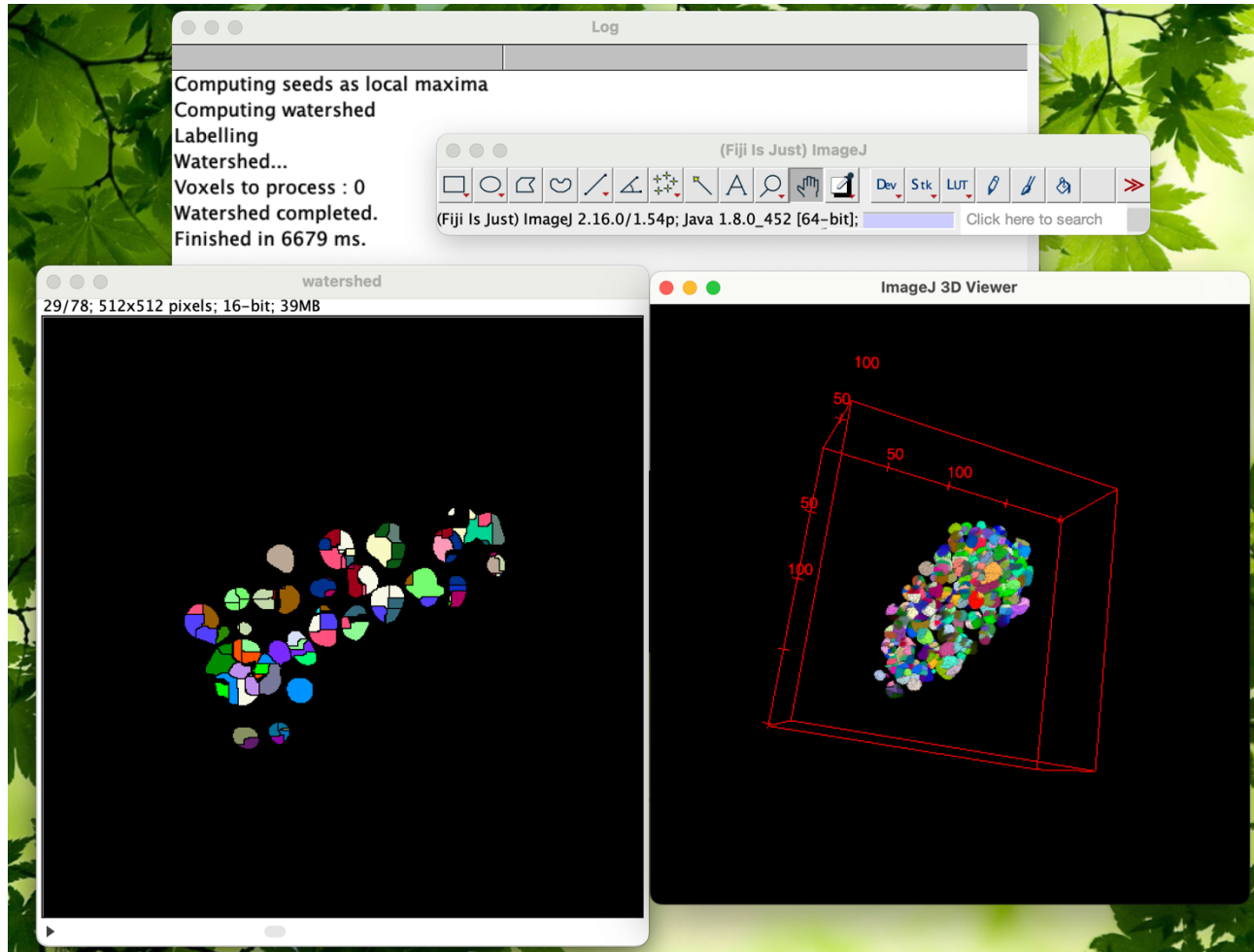


Figure 6: Screenshot of 3D nuclei segmentation performed in ImageJ using the 3D Suite Segmentation plugin. The plugin automatically computed seed points and applied watershed segmentation to separate connected nuclei (log window, top). The resulting segmentation and 3D visualization are shown in the left and right panels, respectively.

To compare the custom Python workflow with an established image analysis platform, the same blastocyst dataset was segmented using the 3D Suite Segmentation plugin in ImageJ. As shown in Figure 6, the plugin automatically computed local maxima as seed points before performing watershed segmentation. The resulting segmentation produced multiple fragments within many individual nuclei, indicating a high degree of over-segmentation. This effect can be visualized in 3D manner, with 3D Viewer plugin, where numerous nuclei appear subdivided into several differently colored regions in their surface.

3.6. Evaluation of 3D mouse blastocyst segmentation.

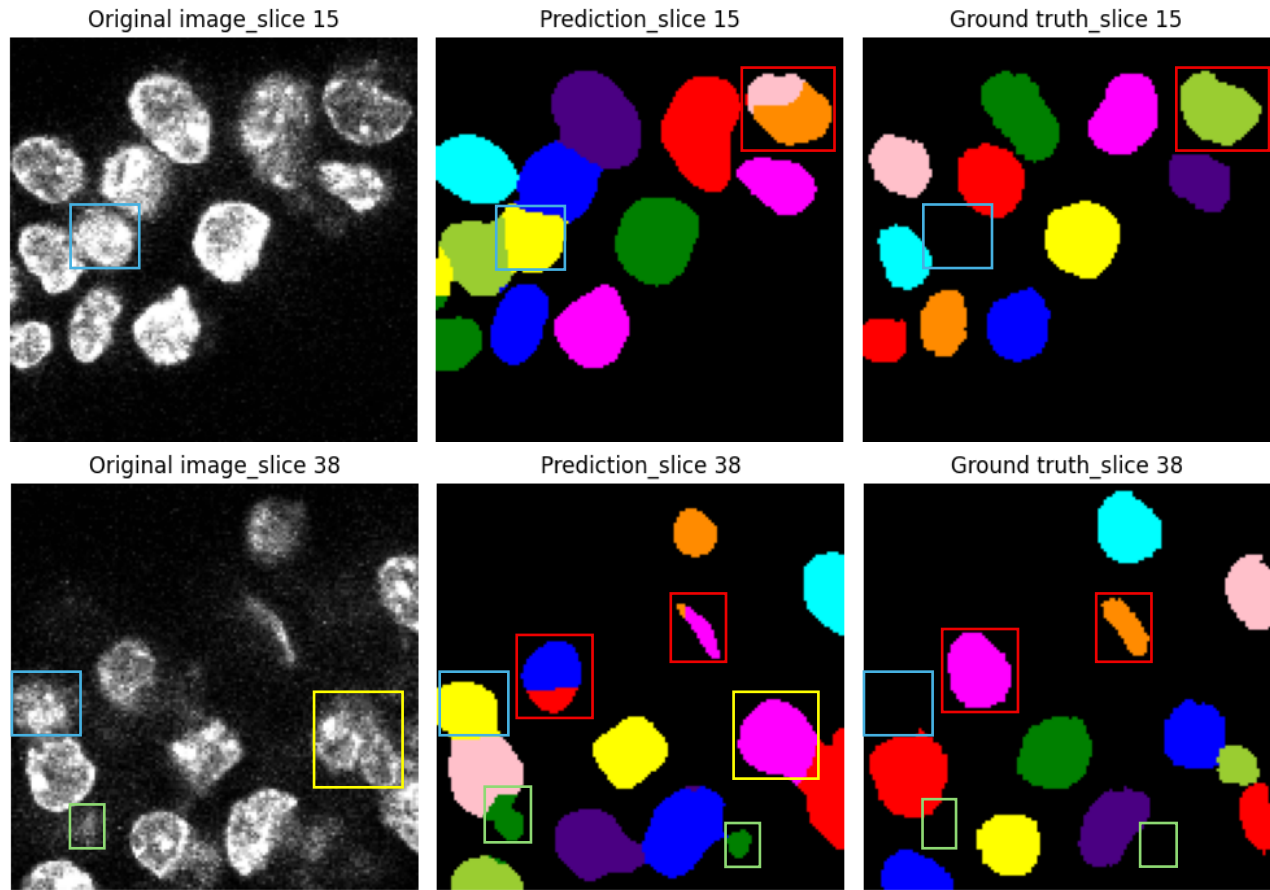


Figure 7: Representative 3D DAPI segmentation results for a cropped image patch (slices 15 and 38). The panels display the original DAPI signal (left), the Python-generated segmentation (middle), and the manually annotated ground truth (right). Ground-truth nuclei were manually segmented and assigned unique labels using the Segmentation Editor plugin in Fiji/ImageJ. Colored boxes highlight discrepancies between the automated pipeline and the manual annotations. Red boxes indicate over-segmentation, where a single nucleus was incorrectly divided into multiple objects. Blue boxes highlight nuclei that were visible in the original microscope image but absent from the ground-truth annotation. Yellow boxes indicate regions containing closely adjacent nuclei with poorly defined boundaries. Green boxes indicate image artifacts that were incorrectly identified as nuclei by the automated pipeline but would typically be excluded during manual annotation.

Figure 7 illustrates several representative types of disagreement between the predicted segmentation and the manual annotation ground truth, marked by boxes. The missing errors marked in blue box likely resulted from the interpolation process in annotation workflow, in which nuclei

were manually labeled on selected slices and interpolated across the z-stack. Although such missing labels have little impact on the total nuclei count, they introduce inaccuracies into the reference segmentation. In these cases, both automated segmentation and manual annotation are challenging because neighboring nuclei cannot be clearly distinguished. Together, these examples demonstrate that inconsistencies between prediction and ground truth arise both from automatic algorithm and manual annotation.

In addition, the proposed segmentation pipeline achieved a precision of 0.686, recall of 0.913, and F1 score of 0.783. The high recall indicates that most nuclei present in the image stack were successfully detected by the automated workflow, suggesting that the pipeline was effective at minimizing missed nuclei. However, the lower precision indicates the presence of false-positive detections, which can be attributed to several factors observed during visual inspection of the segmentation results mentioned above.

4. Discussion

The aim of this project is to develop and evaluate nuclei segmentation methods for both 2D and 3D fluorescence microscopy images. Using Gaussian filtering, Otsu thresholding, morphological operations, and watershed segmentation, the workflows were implemented in both ImageJ/Fiji and Python as shown in Figure 1. Segmentation performance was evaluated through both qualitative visual evaluation and quantitative comparison with manually annotated ground-truth data.

In the 2D mouse brain dataset, $\sigma = 1$ provided the best compromise between noise reduction and preservation of nuclear boundaries. The results in Figure 2 demonstrate that the Gaussian smoothing parameter strongly influences segmentation performance. Since σ controls the spatial extent of the kernel, it determines the degree of image blurring. A larger σ produces a wider Gaussian kernel, leading to stronger averaging of pixel intensities over a larger neighborhood. This reduces noise but can also weaken nuclear boundaries and result in under-segmentation. Therefore, additional morphological operations were applied to mitigate segmentation errors. In general, the performance of this segmentation workflow depended on the spatial arrangement of nuclei. In regions with relatively isolated and regularly shaped nuclei, the algorithm produced results that closely matched visual observations. However, performance decreased in densely packed regions where nuclei exhibited irregular morphologies and extensive overlap. These limitations arise

because classical intensity-based segmentation methods rely on local contrast and shape assumptions, which become less reliable when neighboring nuclei are closely clustered, consistent with the review by Rahman (2025) [18].

For the 3D blastocyst dataset, watershed segmentation effectively separated these structures into individual nuclei and produced a final count of 148 nuclei. Although the correct nuclei count number could not be verified, this value falls within the reported range for late-stage mouse blastocysts, which typically contain approximately 90–150 cells prior to implantation, as described by Saiz et al. (2016) [19]. Therefore, the estimated nuclei count appears biologically sensible. Overall, the segmentation result appears agreeable from the exterior view (Figure 5B); however, internal inspection (Figure 5C) reveals that the segmentation is not fully consistent, with instances of multiple segmented regions within a single nucleus. Besides, comparison with the ImageJ workflow showed that the 3D Suite plugin frequently produced over-segmented nuclei due to its automatic seed generation procedure (Figure 6). To improve segmentation performance using ImageJ plugins, several preprocessing parameters, including smoothing, filtering, and thresholding, can be further optimized through parameter tuning.

From Figure 7, several challenges in the segmentation task are apparent, arising both from an automated workflow and from the validation process itself. Multiple types of discrepancies were observed, including over-segmentation of individual nuclei, difficulties in separating closely adjacent nuclei, false-positive detections caused by imaging artifacts, and missing labels in the ground-truth annotations. Furthermore, manual annotation is itself a challenging and subjective process that requires expert knowledge and may not always accurately reflect the true biological structures. Consequently, some detections classified as false positives by comparison with the ground truth may in fact correspond to real nuclei that were absent during manual annotation. Such annotation errors can affect quantitative performance metrics, including precision, recall, and the F1 score. One way to improve this is to have the ground truth labeled by several experts, which has been done in the literature.

Generally, ImageJ/Fiji remains one of the most widely used platforms for biological image analysis due to its user-friendly graphical interface, which enables image processing without programming experience. The software offers fast image visualization, manual annotation tools, and a wide range of built-in plugins for segmentation and 3D image analysis. These features make ImageJ

particularly useful for rapid exploratory analysis. However, several limitations were observed in this study. Although the available 3D segmentation plugins were straightforward to use, they did not achieve the desired segmentation quality for the blastocyst dataset. In contrast, the custom Python pipeline provided greater flexibility, reproducibility, and automation, making it more suitable for high-throughput image analysis.

To further improve segmentation performance, a wide range of algorithms, including machine learning and deep learning approaches, have been developed such as StarDist [20] and Cellpose[21]. These methods offer powerful alternatives for all types, shape of cell conditions. However, these approaches typically require large, annotated training datasets and substantial computational resources. In comparison, the classical Python pipeline developed in this study is computationally efficient and highly interpretable, as each processing step can be directly inspected and modified.

5. Conclusion

This project successfully implemented, evaluated, and compared classical image-processing workflows for nuclei segmentation in both 2D and 3D fluorescence microscopy datasets using ImageJ/Fiji and Python. The results demonstrate that Gaussian filtering, Otsu thresholding, morphological processing, and watershed segmentation can effectively identify and separate nuclei when appropriate parameters are selected. However, challenges such as over-segmentation, densely packed nuclei, and imaging artifacts highlight the limitations of classical methods. In addition, manual ground-truth annotation is subjective and may contain inconsistencies, which can affect quantitative evaluation.

Code availability

The corresponding GitHub repository can be accessed via the following link:

<https://github.com/ng-diep/Classical-nuclei-segmentation>

Acknowledgment

This report has taken advantage of ChatGPT primarily for English polishing, scientific phrasing, and improving clarity.

References

1. Hickey, S.M., et al., *Fluorescence Microscopy-An Outline of Hardware, Biological Handling, and Fluorophore Considerations*. Cells, 2021. **11**(1).
2. Kalinin, A.A., et al., *3D Shape Modeling for Cell Nuclear Morphological Analysis and Classification*. Scientific Reports, 2018. **8**(1): p. 13658.
3. Botchkarev, V.A., et al., *Epigenetic Regulation of Gene Expression in Keratinocytes*. Journal of Investigative Dermatology, 2012. **132**(11): p. 2505-2521.
4. Pérez-Ventoe, A., et al., *The responsive nucleus: morphological signatures of cellular state*. Nucleus, 2026. **17**(1): p. 2630145.
5. Baral, S. and M.P. Paing, *Instance segmentation of cells and nuclei from multi-organ cross-protocol microscopic images*. Quant Imaging Med Surg, 2024. **14**(9): p. 6204-6221.
6. Chen, Y., et al., *A Systematic Review on Cell Nucleus Instance Segmentation*. IET Image Processing, 2025. **19**(1): p. e70129.
7. de Lange, F., et al., *Cell biology beyond the diffraction limit: near-field scanning optical microscopy*. J Cell Sci, 2001. **114**(Pt 23): p. 4153-60.
8. Ma, J., et al. *3D Nucleus Instance Segmentation for Whole-Brain Microscopy Images*. in *Information Processing in Medical Imaging*. 2021. Cham: Springer International Publishing.
9. Caicedo, J.C., et al., *Evaluation of Deep Learning Strategies for Nucleus Segmentation in Fluorescence Images*. Cytometry A, 2019. **95**(9): p. 952-965.
10. Rueden, C.T., et al., *ImageJ2: ImageJ for the next generation of scientific image data*. BMC Bioinformatics, 2017. **18**(1): p. 529.
11. Schindelin, J., et al., *Fiji: an open-source platform for biological-image analysis*. Nat Methods, 2012. **9**(7): p. 676-82.
12. *Gaussian Filter*, in *Computational Surface and Roundness Metrology*, B. Muralikrishnan and J. Raja, Editors. 2009, Springer London: London. p. 33-38.
13. Otsu, N., *A Threshold Selection Method from Gray-Level Histograms*. IEEE Transactions on Systems, Man, and Cybernetics, 1979. **9**(1): p. 62-66.
14. van der Walt, S., et al., *scikit-image: image processing in Python*. PeerJ, 2014. **2**: p. e453.
15. Virtanen, P., et al., *SciPy 1.0: fundamental algorithms for scientific computing in Python*. Nature Methods, 2020. **17**(3): p. 261-272.
16. Harris, C.R., et al., *Array programming with NumPy*. Nature, 2020. **585**(7825): p. 357-362.
17. Chiu, C.-L., N. Clack, and c. the napari, *napari: a Python Multi-Dimensional Image Viewer Platform for the Research Community*. Microscopy and Microanalysis, 2022. **28**(S1): p. 1576-1577.
18. Rahman, M.M., et al., *A Systematic Review on Advancement of Image Segmentation Techniques for Fault Detection Opportunities and Challenges*. Electronics, 2025. **14**(5): p. 974.
19. Saiz, N., et al., *Asynchronous fate decisions by single cells collectively ensure consistent lineage composition in the mouse blastocyst*. Nat Commun, 2016. **7**: p. 13463.
20. Stevens, M., et al., *StarDist Image Segmentation Improves Circulating Tumor Cell Detection*. Cancers (Basel), 2022. **14**(12).
21. Stringer, C., et al., *Cellpose: a generalist algorithm for cellular segmentation*. Nature Methods, 2021. **18**(1): p. 100-106.